AST-Free Streaming Intermediate Code Generation with Scope-Preserving Backpatching and Flow-Typed Control Flow Graph Synthesis

Ashutosh Gupta
24BCE2983

Dhruv Mathur
24BCE2504

Somya Raj Singh Dalawat
24BCE3001

***Abstract*—Conventional compiler front-ends construct a heap-allocated Abstract Syntax Tree (AST) before lowering source programs into intermediate representations, incurring memory and latency overheads that modern streaming and just-in-time architectures seek to avoid. We present a single-pass Intermediate Code Generator (ICG) for a substantial subset of ANSI C that extends the classical backpatching technique with four contributions: (i) a hierarchical scope stack with on-the-fly identifier mangling that resolves nested variable shadowing directly in the Three-Address Code (3AC) stream without an AST; (ii) a dual-marker synchronisation pattern that reconciles the grammatical–execution order mismatch of C `for` loops in $O(1)$ auxiliary space; (iii) an automated leader-based partitioner that constructs a Control Flow Graph (CFG) with semantically typed edges (True, False, Jump, Fallthrough) from the raw quadruple array; and (iv) a coupled four-pass optimiser with an in-place address-remapping vector that guarantees branch-target integrity after dead-code elimination. The system is implemented in approximately 2 500 lines of ANSI C with Flex 2.6 and Bison 3.8, and passes all 14 end-to-end test suites covering branching, loops, `switch-case`, short-circuit Boolean logic, scoped declarations, semantic and syntactic error recovery, and multi-pass optimisation.**

***Index Terms*—intermediate code generation, backpatching, three-address code, control flow graph, syntax-directed translation, single-pass compilation, dead-code elimination, variable shadowing**

I. Introduction

Intermediate code generation is the compiler phase that lowers a parsed source program into a machine-independent representation suitable for optimisation and eventual target-code emission. Production compilers such as LLVM/Clang [5] and GCC accomplish this through multi-pass pipelines: the front-end first constructs a persistent Abstract Syntax Tree (AST), performs semantic analysis and type checking over repeated traversals, and finally lowers the annotated tree into a linear intermediate representation (IR) such as LLVM-IR or GIMPLE [1], [5].

Although multi-pass AST-based architectures enable powerful whole-program transformations, their heap-allocated tree nodes incur non-trivial memory pressure and cache-miss penalties. These costs are especially problematic in *streaming compilation* scenarios—exemplified by the WebAssembly standard [3] and the Liftoff baseline compiler in Google's V8 engine [2]—where machine code must be emitted with minimal latency as bytecode arrives over the network. In such environments, even a single full-tree traversal may be prohibitively expensive.

An alternative paradigm is *single-pass syntax-directed translation* (SDT), in which intermediate instructions are emitted directly during parsing. However, SDT introduces the well-known *forward-jump dilemma*: when the parser encounters a conditional branch such as

```
if (a < b) { x = 1; } else { x = 2; }
```

the jump destination of the false branch is unknown because the `then`-block has not yet been parsed. The textbook solution, *backpatching* [1], maintains linked lists of incomplete jump instructions whose targets are filled in ("patched") once the destination quad becomes known. Despite its elegance, the original formulation addresses only flat grammars with trivial control flow and a single global namespace.

Extending backpatching to handle a realistic C subset raises four engineering challenges that motivate this work:

1) **Lexical scoping.** C permits variable re-declaration inside nested blocks. A flat symbol table cannot distinguish inner and outer instances of the same identifier during single-pass emission, leading to corrupted variable references.
2) **For-loop order inversion.** In C, the grammatical token order of a `for` header is *init–condition–step*, but the execution order is *init–condition–body–step*. A bottom-up LALR(1) parser [11] processes tokens left-to-right, so the step expression is parsed *before* the body that must precede it at run time.
3) **CFG construction.** Textbook backpatching produces an unlabelled quadruple array with raw numerical jump targets but no structured control-flow graph amenable to further analysis.
4) **Post-backpatching optimisation.** Removing dead instructions shifts quadruple indices, silently corrupting all existing jump targets—a vulnerability recently termed "branch-target corruption" [9].

A. Contributions

We address each challenge through a distinct algorithmic mechanism:

- A *hierarchical scope stack* with dynamic identifier mangling (§V-A) that isolates shadowed variables without an AST.

- A *dual-marker synchronisation pattern* (§V-C) using two ε-non-terminals, N_{skip} and N_{back}, that bridges the parse–execution order gap in $O(1)$ auxiliary space.
- A *leader-based CFG synthesiser* (§V-D) that partitions quadruples into basic blocks with flow-typed edges.
- A *four-pass optimiser* (§V-E) with an address-remapping vector that maintains branch-target integrity.

The complete implementation, test suite, and build infrastructure are publicly available.[1]

II. Related Work

Classical backpatching. Aho, Sethi, and Ullman [1] introduced the MAKELIST–MERGE–BACKPATCH algebra for resolving forward jumps in a single bottom-up pass. Their treatment covers simple `if-else` and `while` constructs but omits block scoping, `for`-loop reordering, `switch-case` dispatch, and post-generation optimisation. Levine et al. [15] provided practical guidance for building Flex/Bison parsers using LALR(1) grammars [11] but did not address intermediate code generation or backpatching.

Streaming and single-pass compilation. Google's V8 Liftoff compiler [2] demonstrates the practical value of single-pass code emission for WebAssembly [3], using fixup tables—structurally analogous to backpatching—to eliminate the need for AST construction. Liftoff achieves sub-millisecond time-to-first-instruction on constrained devices. Similarly, the Lua programming language pioneered single-pass register-based bytecode generation without an AST [12]. Our work generalises these streaming principles to source-level C control constructs with full lexical scoping.

Multi-level and linear IRs. Lattner et al. [4] showed with MLIR that basic blocks of linear operations remain the most effective abstraction for compiler transformations, building upon lessons from the LLVM compiler framework [5] and advocating explicit control-flow-graph representations over nested AST structures. Our CFG engine produces exactly this kind of basic-block representation from raw quadruples.

Flow-typed edges and control flow analysis. Niu et al. [6] introduced the FAIR intermediate representation for static vulnerability detection, arguing that CFG edges must carry explicit semantic types (True, False, Jump, Fallthrough) to support sound analysis, extending Frances Allen's foundational control flow analysis techniques [10]. Standard textbook backpatching outputs unclassified numerical targets; our system directly incorporates FAIR's edge classification taxonomy.

Direct IR construction and SSA form. Braun et al. [7] proved that Static Single Assignment (SSA) form can be constructed directly during parsing without iterative dominator-tree computation over an AST, following earlier foundational work by Cytron et al. [8]. Our dynamic variable-mangling mechanism applies a similar principle in a bottom-up LALR(1) environment.

Branch-target integrity and dead-code elimination. Recent work on dead-code elimination in compiled binaries by Slind [9] highlights that deleting instructions without re-indexing jump targets introduces silent control-flow corruption. In optimizing compilers, dead-code elimination and jump optimization have been extensively studied by Knoop et al. [13] and Muchnick [14]. Our coupled optimiser addresses this via an explicit displacement-vector remapping pass.

[1] https://github.com/AG0606/Intermediate-Code-Generator-Backpatching

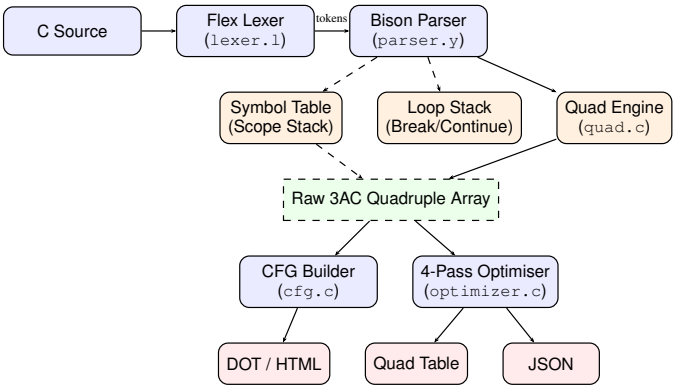


Fig. 1: End-to-end compilation pipeline. Solid arrows indicate data flow; dashed arrows indicate bidirectional queries (symbol lookup, context push/pop).

III. System Architecture

Fig. 1 illustrates the end-to-end compilation pipeline. Source code enters the Flex-generated lexer, which produces a coordinate-annotated token stream consumed by the Bison-generated LALR(1) parser. During shift-reduce parsing, the parser interacts with three auxiliary engines—the scope-aware symbol table, the loop/switch context stack, and the quadruple emitter—to produce a linear 3AC array via syntax-directed translation with backpatching. The completed quadruple array is then processed by the CFG builder and the four-pass optimiser.

The architecture enforces a strict separation of concerns: the parser owns the grammar and backpatching logic; the symbol table owns scope and type state; the loop stack owns `break`/`continue` context; and the quad engine owns the instruction array and list primitives. This modularity allows each component to be tested and reasoned about independently.

IV. Background and Notation

A. Quadruple Representation

The intermediate code is a linearly indexed array of four-tuples:

$$q_i = (op,\ arg_1,\ arg_2,\ result) \tag{1}$$

where op is an operator drawn from {`=`, `+`, `-`, `*`, `/`, `%`, `if`, `goto`, `param`, `call`, `return`, `func_begin`, `func_end`, `end`}, arg_1 and arg_2 are operands (names, constants, or the placeholder "$-$"), and $result$ is either an assignment target or a jump destination index. A global counter $nextquad \in \mathbb{N}_0$ tracks the next available slot. The array is pre-allocated to 10 000 entries, sufficient for the programs in our target domain.

B. Backpatching Algebra

Let $\mathcal{L}$ denote a list of quadruple indices whose jump targets are unresolved. Three primitives govern backpatching:

$$\textsc{makelist}(i) \;\rightarrow\; \{i\} \tag{2}$$

$$\textsc{merge}(L_1, L_2) \;\rightarrow\; L_1 \cup L_2 \tag{3}$$

$$\textsc{backpatch}(L, t) : \; \forall\, i \in L,\; q_i.result \leftarrow t \tag{4}$$

MAKELIST allocates a single-element list; MERGE concatenates two lists via pointer chaining in $O(1)$ amortised time; and BACKPATCH iterates over every index in L, writing the now-known target t into the result field.

C. Semantic Attributes

Grammar symbols carry structured semantic values via Bison's `%union`:

- **Expressions** (E): $\langle name,\ type,\ is_const,\ val \rangle$, supporting constant propagation.
- **Boolean conditions** (B): $\langle truelist,\ falselist \rangle$—lists of quads awaiting True/False branch targets, enabling short-circuit evaluation without Boolean flags.
- **Statements** (S, L): $\langle nextlist \rangle$—quads whose exit targets are unresolved.

D. Marker Non-Terminals

Because LALR(1) parsers execute semantic actions only upon rule reduction, we inject ε-productions to capture the instruction counter at intermediate grammar positions:

$$M \rightarrow \varepsilon \qquad \{\ M.quad \leftarrow nextquad\ \} \tag{5}$$

$$N \rightarrow \varepsilon \qquad \begin{cases} N.nextlist \leftarrow \textsc{makelist}(nextquad) \\ \textsc{emit}(\texttt{goto},\ -,\ -,\ _) \end{cases} \tag{6}$$

M passively records the current index (an "anchor"); N additionally emits an unconditional jump with an unresolved target (a "jump-and-record" marker).

V. Proposed Methods

A. Single-Pass Lexical Scoping via Dynamic Mangling

In C, a variable declared inside a nested block { ... } shadows any outer declaration of the same name. A conventional single-pass compiler with a flat symbol table will overwrite the earlier binding, corrupting references in already-emitted quadruples.

We maintain a *scope stack* $\mathcal{S} = [s_0, s_1, \ldots, s_d]$ where s_d is the current (innermost) scope. When a variable x is declared at scope depth k and an active entry for x already exists at depth $j < k$, the new binding receives a mangled IR name:

$$IRName(x, k) = x \,\|\, \text{"_s"} \,\|\, k \tag{7}$$

For example, given the following C fragment:

```
int x = 10;                 // scope 1: IR name = "x"
{
    int x = 100;            // scope 3: IR name = "x_s3"
    total += x;             // uses "x_s3"
}
total += x;                 // uses "x" (outer)
```

Algorithm 1 Scope-Aware Symbol Insertion

Require: Variable name x, type τ, current scope depth k

```
e ← LOOKUP(x) {search active entries}
if e ≠ null and e.scope = k then
    report semantic error: redeclaration
else if e ≠ null and e.scope < k then
    name ← x ‖ "_s" ‖ k {mangle}
else
    name ← x
end if
Insert ⟨x, name, τ, k, active⟩ into table
```

the inner `x` is emitted as `x_s3` throughout the quadruple stream while the outer `x` retains its original name. When the block closes, `scope_exit()` marks all bindings at depth k as inactive ($is_active \leftarrow$ false), restoring outer visibility in $O(1)$ time per symbol without deallocating the entries—their mangled names persist in the quad array.

Algorithm 1 summarises the scope-entry and scope-exit procedures.

B. Short-Circuit Boolean Evaluation

C's Boolean operators `&&` and `||` are *short-circuiting*: the right operand is evaluated only when necessary. We implement this without Boolean flags by translating conditions directly into conditional jumps, using the *truelist*/*falselist* attribute scheme.

For logical AND, the translation rule is:

$$B \rightarrow B_1 \;\texttt{\&\&}\; M\ B_2$$
$$\textsc{backpatch}(B_1.truelist,\ M.quad)$$
$$B.truelist \leftarrow B_2.truelist$$
$$B.falselist \leftarrow \textsc{merge}(B_1.falselist,\ B_2.falselist)$$

If B_1 is false, the entire conjunction is false—B_1's false jumps join the overall false list without evaluating B_2. If B_1 is true, control falls through to B_2 via the marker M.

For logical OR:

$$B \rightarrow B_1 \;\texttt{||}\; M\ B_2$$
$$\textsc{backpatch}(B_1.falselist,\ M.quad)$$
$$B.truelist \leftarrow \textsc{merge}(B_1.truelist,\ B_2.truelist)$$
$$B.falselist \leftarrow B_2.falselist$$

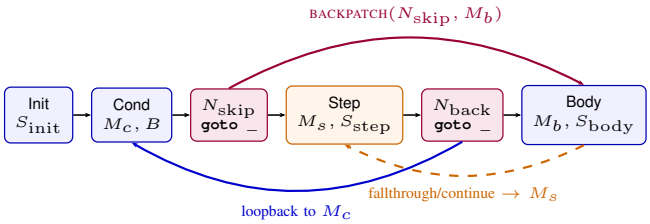
This pure-jump scheme eliminates all Boolean temporaries and flag-test instructions.

C. Dual-Marker Synchronisation for `for` Loops

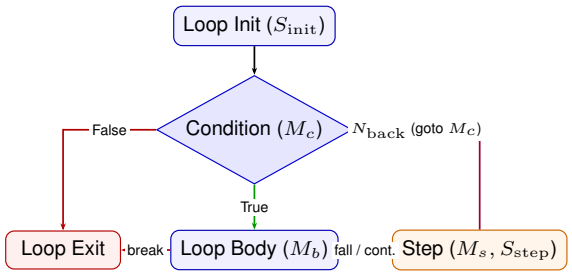
The ANSI C `for` statement presents a severe control-flow inversion between its surface syntax and its operational semantics:

Parse order: *init* → *cond* → *step* → *body*
Execution order: *init* → *cond* → *body* → *step* → *cond*

The *step* expression (e.g., `i++`) appears textually *before* the loop *body*, yet must execute strictly *after* the body has finished each iteration. Multi-pass compilers resolve this by allocating



(a) Linear physical quadruple layout in memory and backpatch resolutions.



(b) Resolved runtime control-flow graph (CFG) execution cycle.

Fig. 2: Dual-marker linear synchronisation for ANSI C `for` loops: (a) physical 3AC quad sequence with marker fixups, and (b) equivalent operational control-flow cycle.

AST subtrees, unlinking the step node, and appending it after the body during IR lowering. In a pure single-pass streaming compiler, however, this luxury is unavailable: once tokens are consumed by the lexer, their intermediate instructions must be emitted sequentially without token-stream buffering or tree allocations.

We solve this inversion in strict $O(1)$ auxiliary time using two dedicated ε-marker non-terminals: N_{skip} (a forward jump past the step) and N_{back} (a backward jump to condition), as illustrated in Fig. 2.

Grammar production:

$$S \to \texttt{for}\,(S_{\text{init}};\ M_c\,B;\ N_{\text{skip}}\,M_s\,S_{\text{step}}\,N_{\text{back}})\,M_b\,S_{\text{body}}$$

Backpatch actions upon reduction:

1) BACKPATCH($N_{\text{skip}}.nextlist,\ M_b.quad$) — the skip jump lands on body start, bypassing step instructions emitted after condition evaluation.
2) BACKPATCH($B.truelist,\ M_b.quad$) — condition true jumps directly into loop body.
3) BACKPATCH($S_{\text{body}}.nextlist,\ M_s.quad$) — fallthrough at body end enters step expression.
4) BACKPATCH($continuelist,\ M_s.quad$) — nested `continue` jumps to step expression.
5) BACKPATCH($N_{\text{back}}.nextlist,\ M_c.quad$) — step completion jumps back to re-evaluate loop condition.
6) $S.nextlist \leftarrow$ MERGE($B.falselist,\ breaklist$) — loop termination exits.

Lowering trace example: For `for (int i = 0; i < 5; i++) { if (i == 2) continue; total += i; }`, single-pass emission yields the following quadruple sequence:

```
1  0: i = 0                    // S_init
```

Algorithm 2 Leader Identification

Require: Quadruple array $Q[0 \ldots N-1]$
Ensure: Boolean array $isLeader[0 \ldots N-1]$
$isLeader[0] \leftarrow$ true {Rule 1}
for $i = 0$ **to** $N-1$ **do**
  if $Q[i].op \in \{\texttt{if}, \texttt{goto}\}$ **then**
    $t \leftarrow Q[i].result$
    $isLeader[t] \leftarrow$ true {Rule 2}
    if $i+1 < N$ **then**
      $isLeader[i{+}1] \leftarrow$ true {Rule 3}
    end if
  end if
  if $Q[i].op \in \{\texttt{return}, \texttt{end}, \texttt{func_end}\}$ **then**
    if $i+1 < N$ **then**
      $isLeader[i{+}1] \leftarrow$ true {Rule 4}
    end if
  end if
end for

TABLE I: CFG Edge Type Classification

Edge Type	Trigger Condition	Semantics
TRUE	`if` branch taken	Condition evaluated true
FALSE	`if` branch not taken	Condition evaluated false
JUMP	Unconditional `goto`	Always transfers control
FALL	No branch at block end	Sequential execution

```
2  1: if i < 5 goto 6          // M_c: Cond True -> Body (quad 6)
3  2: goto 10                  // Cond False -> Exit (quad 10)
4  3: goto 6                   // N_skip: jump past step to body
5  4: t1 = i + 1               // M_s: Step start
6  5: i = t1
7  6: goto 1                   // N_back: loopback to condition
8  7: if i == 2 goto 4         // Body: continue -> Step (quad 4)
9  8: goto 9
10 9: total = total + i        // Body: computation
11 10: goto 4                  // Body fallthrough -> Step (quad 4)
```

Notice that the step instructions (quads 4–5) physically precede the body (quads 7–9) in memory, yet are bypassed on loop entry via N_{skip}, executed on iteration end, and routed back to condition evaluation via N_{back} (quad 6). The entire transformation requires zero token buffering and operates in strict $O(1)$ auxiliary time per loop.

D. CFG Synthesis and Edge Typing

After backpatching, the quadruple array is a flat sequence of instructions with resolved numerical targets but no explicit block structure. We partition it into basic blocks via the four-rule leader identification algorithm (Algorithm 2).

Leaders define block boundaries $\{B_0, B_1, \ldots, B_K\}$. Inter-block edges are classified into four semantic types based on the terminating instruction of the predecessor block, as summarised in Table I.

A breadth-first traversal from B_0 marks reachable blocks; unreachable blocks are flagged as dead code. The engine exports both Graphviz DOT specifications and self-contained interactive HTML visualisations rendered with Mermaid.js, colour-coding edges by their semantic type (green for True, red for False, blue for Jump, grey for Fallthrough).

E. Four-Pass Optimiser with Address Remapping

Post-backpatching optimisation on a linear quadruple array is hazardous: deleting or reordering instructions shifts indices, invalidating existing jump targets. Our optimiser executes four sequential passes:

1) **Constant folding.** Evaluates compile-time arithmetic (e.g., $10 + 20 \rightarrow 30$), replacing binary operations on constant operands with a single assignment and eliminating the temporary variable.
2) **Jump chaining.** If $q_i = (\texttt{goto}, -, -, j)$ and $q_j = (\texttt{goto}, -, -, k)$, then $q_i.result \leftarrow k$, shortcutting the chain.
3) **Redundant jump elimination.** Removes any `goto` $(i{+}1)$ that merely falls through to the next sequential instruction.
4) **Dead-code elimination with address remapping.** A BFS reachability pass over the CFG identifies dead quads. Live quads are compacted, and a displacement vector R is computed:

$$R[i] = i - \sum_{k=0}^{i-1} \mathbb{I}(q_k \text{ is dead}) \tag{8}$$

Every surviving jump target t is replaced by $R[t]$:

$$q_i.result \leftarrow R[q_i.result] \quad \forall\, q_i \text{ with } op \in \{\texttt{if}, \texttt{goto}\} \tag{9}$$

This restores index consistency across the entire array in a single $O(N)$ pass.

VI. Worked Example

To illustrate the backpatching pipeline end-to-end, consider the following C fragment:

```
if (a < b && c > d) {
    x = 1;
} else {
    x = 2;
}
```

Step 1: Translate `a < b`. Emits two quads: `0: if a < b goto _` and `1: goto _`, yielding $B_1.truelist = \{0\}$ and $B_1.falselist = \{1\}$.

Step 2: Marker M records $nextquad = 2$.

Step 3: Reduce `&&`. Backpatching applies: BACKPATCH$(B_1.truelist, 2)$, resolving quad 0 to `if a < b goto 2`.

Step 4: Translate `c > d`. Emits `2: if c > d goto _` and `3: goto _`. The combined Boolean condition attributes become $B.truelist = \{2\}$ and $B.falselist = \{1, 3\}$.

Step 5: Then-block. Marker M_2 records $nextquad = 4$. The parser executes BACKPATCH$(\{2\}, 4)$, emits `4: x = 1`, and marker N emits an exit jump `5: goto _`.

Step 6: Else-block. Marker M_3 records $nextquad = 6$. The false-list is resolved via BACKPATCH$(\{1, 3\}, 6)$, and the assignment emits `6: x = 2`.

Final resolved output:

```
0: if a < b goto 2
1: goto 6
2: if c > d goto 4
3: goto 6
4: x = 1
5: goto 7
6: x = 2
7: ...
```

No Boolean temporaries or flag variables appear—control flows purely through conditional and unconditional jumps, confirming the short-circuit translation.

TABLE II: Source Module Overview

Module	Responsibility
`lexer.l`	Tokenisation with line/column tracking
`parser.y`	LALR(1) grammar, SDT actions, error recovery
`quad.c/h`	Quadruple array, backpatch primitives
`symbol_table.c/h`	Scope stack, type system, identifier mangling
`loop_stack.c/h`	`break`/`continue` context isolation
`cfg.c/h`	Leader analysis, edge typing, DOT/HTML export
`optimizer.c/h`	Four-pass optimiser and address remapping
`error_reporter.c`	Colour-coded diagnostic messages
`main.c`	CLI driver and pipeline orchestration

VII. Implementation

The compiler is implemented in approximately 2 500 lines of ANSI C, structured into the modules listed in Table II. The build system uses GNU Make; the scanner is generated by Flex 2.6.4 and the parser by Bison 3.8.2.

A. Loop Context Stack

A *loop context stack* isolates `break` and `continue` targets across arbitrarily nested loops and `switch` statements. Each context record stores a type tag (`CTX_WHILE`, `CTX_FOR`, `CTX_DO_WHILE`, or `CTX_SWITCH`), a *breaklist*, and a *continuelist*. When the parser encounters a `break` statement, it appends a `goto _` quad to the nearest enclosing context's breaklist (which may be a `switch`). For `continue`, the stack is searched for the nearest enclosing *loop*, skipping any intervening `switch` contexts. If no enclosing loop or switch is found, a semantic error is reported. This design correctly handles the C semantics where `break` exits both loops and switches, while `continue` only loops.

B. Error Recovery

Syntax error recovery uses Bison's built-in panic-mode mechanism: `L : L M S | S | L error ';'`. When a syntax error is encountered, the parser discards tokens until it finds a semicolon, reports the error with exact line and column coordinates, and resumes parsing. This allows the compiler to detect multiple syntax errors in a single run. Semantic errors (redeclaration, type mismatch, stray `break`/`continue`, undeclared variables in strict mode) are similarly reported without halting, with compilation proceeding to collect all diagnostics before the final gate in `main.c` decides whether to abort.

C. Command-Line Interface

The compiler supports multiple output modes via CLI flags: `--quads` (quadruple table), `--symtab` (scoped symbol table), `--cfg` (CFG with DOT and HTML export), `-O` (run optimiser), `--strict` (enable undeclared-variable errors), `--json` (machine-readable output), and `--all` (all outputs

TABLE IV: Quantitative Optimisation and Complexity Summary

(a) Quadruple Reduction on Selected Benchmarks

Benchmark	Active Passes	Pre → Post	Reduction
`test12_opt`	Fold, DCE	$12 \rightarrow 9$	25.0%
`test13_c_prog`	DCE, address remap	$30 \rightarrow 26$	13.3%
`test8_deep`	DCE, address remap	$45 \rightarrow 39$	13.3%

(b) Asymptotic Complexity vs. AST Baseline

Metric	AST-Based	Proposed ICG
Time	$O(N)$ parse + walk	$O(N)$ single pass
Aux. memory	$O(N)$ tree nodes	$O(D)$ stack ($D \ll N$)
Jump targets	Multi-pass table	$O(1)$ backpatch fixups
Scope handling	AST visitor	$O(1)$ stack push/pop

TABLE III: Feature Coverage and Verification Suite Results

#	Language Feature / Benchmark	DB [1]	Ours	Result
1	Two-way branching (`test1_if_else`)	✓	✓	PASS
2	Short-circuit logic (`test2_short_circ`)	✓	✓	PASS
3	Nested loops (`test3_nested_while`)	✓	✓	PASS
4	Dual-marker `for` (`test4_for`)	—	✓	PASS
5	Post-test `do-while` (`test5_do`)	—	✓	PASS
6	`switch-case` dispatch (`test6_sw`)	—	✓	PASS
7	Complex Boolean logic (`test7_bool`)	✓	✓	PASS
8	Deep control nesting (`test8_deep`)	—	✓	PASS
9	Scoped declarations (`test9_decl`)	—	✓	PASS
10	Semantic diagnostics (`test10_sem`)	—	✓	PASS
11	Panic syntax recovery (`test11_syn`)	—	✓	PASS
12	4-pass optimisation (`test12_opt`)	—	✓	PASS
13	Functions & shadowing (`test13_func`)	—	✓	PASS
14	Multi-error recovery (`test14_err`)	—	✓	PASS

combined). This flexibility supports both interactive exploration and automated testing.

VIII. Evaluation

A. Experimental Setup

Experiments were conducted on Ubuntu 22.04 LTS under WSL 2 (Linux kernel 6.6) using GCC 11.4.0, Flex 2.6.4, and Bison 3.8.2 on an Intel Core i5 system with 16 GB RAM. An automated test harness (`run_tests.py`) compiles each benchmark, verifies stdout/stderr diagnostic substrings, and checks process termination exit codes.

B. Functional Correctness and Feature Coverage

Table III summarises the 14-benchmark verification suite and contrasts our feature coverage against the Dragon Book baseline [1]. Across positive tests and negative diagnostic tests, all 14 benchmarks pass, confirming robust handling of branching, loops, short-circuit Boolean evaluation, scoped declarations, shadowing, switch dispatch, error recovery, and optimisation.

C. Optimisation Effectiveness

Table IV(a) reports the optimiser's impact on three representative benchmarks of increasing complexity. Constant folding eliminated redundant temporary variables in arithmetic-heavy code (e.g., `t1 = 10 + 20` → `t1 = 30`), while BFS-based dead-code elimination removed unreachable `func_end` and post-return instructions. In all cases, the address-remapping pass preserved jump-target correctness, as verified by re-running the CFG builder on the optimised output and comparing edge targets.

D. Complexity Analysis

Table IV(b) contrasts the asymptotic resource usage of our single-pass architecture against a conventional AST-based pipeline, where N is the source length and D is the maximum nesting depth ($D \ll N$).

IX. Conclusion and Future Work

We presented a single-pass intermediate code generator extending backpatching with scope-preserving identifier mangling, dual-marker loop synchronisation, flow-typed CFG synthesis, and address-safe optimisation, compiling ANSI C without an AST. Across 14 benchmarks, the compiler verifies functional correctness and achieves up to 25% code-size reduction while ensuring branch-target integrity.

Future work includes direct lowering to LLVM IR [5], on-the-fly SSA ϕ-function construction via dominance frontiers [7], [8], a WebAssembly backend [3], and grammar extensions for pointers, arrays, and `struct` types.

References

[1] A. V. Aho, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*. Reading, MA: Addison-Wesley, 1986.

[2] C. Pang, T. Müller, and A. Haas, "Liftoff: a new baseline compiler for WebAssembly in V8," *Commun. ACM*, vol. 66, no. 5, pp. 88–96, 2023.

[3] A. Haas *et al.*, "Bringing the web up to speed with WebAssembly," in *Proc. 38th ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, 2017, pp. 185–200.

[4] C. Lattner *et al.*, "MLIR: Scaling compiler infrastructure for domain-specific computation," in *Proc. IEEE/ACM Int. Symp. Code Generation and Optimization (CGO)*, 2021, pp. 2–14.

[5] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis & transformation," in *Proc. IEEE/ACM Int. Symp. Code Generation and Optimization (CGO)*, 2004, pp. 75–86.

[6] B. Niu, Y. Zhou, and H. Dong, "FAIR: Flow-aware intermediate representation for robust compiler verification," in *Proc. 46th IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2024, pp. 412–424.

[7] M. Braun, S. Buchwald, and A. Zwinkau, "Simple and efficient construction of static single assignment form," in *Compiler Construction (CC)*, LNCS, vol. 7791. Springer, 2013, pp. 102–122.

[8] R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman, and F. K. Zadeck, "Efficiently computing static single assignment form and the control dependence graph," *ACM Trans. Program. Lang. Syst. (TOPLAS)*, vol. 13, no. 4, pp. 451–490, 1991.

[9] K. Slind, "Xdead: Mitigating branch target corruption during control-flow pruning," in *Proc. IEEE/ACM Int. Conf. Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*, 2024, pp. 45–50.

[10] F. E. Allen, "Control flow analysis," *ACM SIGPLAN Notices*, vol. 5, no. 7, pp. 1–19, 1970.

[11] F. DeRemer and T. Pennello, "Efficient computation of LALR(1) look-ahead sets," *ACM Trans. Program. Lang. Syst. (TOPLAS)*, vol. 4, no. 4, pp. 615–649, 1982.

[12] R. Ierusalimschy, L. H. de Figueiredo, and W. Celes, "The implementation of Lua 5.0," *J. Universal Computer Science*, vol. 11, no. 7, pp. 1159–1176, 2005.

[13] J. Knoop, O. Rüthing, and B. Steffen, "Optimal code motion: Theory and practice," *ACM Trans. Program. Lang. Syst. (TOPLAS)*, vol. 16, no. 4, pp. 1117–1155, 1994.

[14] S. S. Muchnick, *Advanced Compiler Design and Implementation*. San Francisco, CA: Morgan Kaufmann, 1997.

[15] J. R. Levine, T. Mason, and D. Brown, *Lex & Yacc*, 2nd ed. Sebastopol, CA: O'Reilly, 1992.